

LuckyHub 高并发营销抽奖与权益发放平台

需求分析与总体设计文档

1. 文档信息

项目	内容
项目名称	LuckyHub 高并发营销抽奖与权益发放平台
英文名称	LuckyHub Marketing Lottery Platform
项目简称	LuckyHub
文档类型	需求分析与总体设计文档
项目类型	Java 后端实战项目
当前版本	V1.0
开发模式	单体模块化架构
目标周期	1~2 周
主要用户	活动用户、运营人员、系统管理员

2. 项目概述

2.1 项目背景

电商、内容平台、会员系统和游戏运营中，经常需要通过抽奖、优惠券、积分、会员权益等方式开展营销活动。

在活动开始、节日促销、新用户拉新等场景中，大量用户可能在短时间内同时参与活动，系统需要处理以下问题：

- 瞬时高并发请求；
- 奖品库存有限；
- 同一用户重复点击；
- 网络重试导致重复中奖；
- 热门活动配置被频繁查询；
- Redis 与数据库之间的数据一致性；
- 活动权限和后台管理；
- 抽奖记录与权益发放的可靠保存。

LuckyHub 是一个面向营销运营场景的高并发抽奖与权益发放平台。系统支持运营人员配置营销活动、奖品、库存、中奖权重和参与规则，普通用户可以参与活动并查询中奖结果，系统管理员负责用户、角色和权限管理。

2.2 项目定位

LuckyHub 第一阶段定位为：

一个采用 Spring Boot、MySQL、Redis 和 Redisson 开发的单体模块化营销活动系统，重点解决高并发抽奖、奖品库存、接口幂等、缓存和权限管理问题。

第一阶段不以实现完整商业平台为目标，而是验证以下后端能力：

- 独立完成数据库设计；
- 独立完成接口设计；
- 使用 Spring Boot 开发后台系统；
- 使用 Redis 解决缓存和并发问题；
- 使用 MySQL 事务保证核心数据一致性；
- 使用 JWT 和 Redis 实现登录状态；
- 使用 RBAC 实现权限控制；
- 使用 Docker Compose 完成一键部署；
- 对高并发抽奖接口进行压测和结果验证。

2.3 项目目标

业务目标

1. 支持运营人员创建和管理营销抽奖活动。
2. 支持配置活动奖品、库存和中奖权重。
3. 支持用户参与单次抽奖和批量抽奖。
4. 支持限制用户每日参与次数。
5. 支持查询抽奖记录 and 用户已获得权益。
6. 支持后台查询活动参与情况和库存消耗情况。
7. 支持用户、角色和权限管理。

技术目标

1. 使用 Redis 缓存热门活动配置。
 2. 使用缓存空值降低缓存穿透风险。
 3. 使用 Redis Lua 脚本原子扣减奖品库存。
 4. 使用请求唯一标识实现接口幂等。
 5. 使用 Redisson 防止同一用户并发重复参与。
 6. 使用 MySQL 唯一索引作为幂等兜底。
 7. 使用 MySQL 事务保存抽奖订单、中奖记录和用户权益。
 8. 使用 Docker Compose 一键启动应用、MySQL 和 Redis。
 9. 使用压力测试验证库存不超卖和请求不重复处理。
-

3. 项目范围

3.1 第一阶段包含内容

第一阶段必须实现：

- 用户登录和退出；
 - JWT 身份认证；
 - Redis 登录状态；
 - 用户、角色、权限管理；
 - 奖品管理；
 - 营销活动管理；
 - 活动奖品和中奖权重配置；
 - 奖品库存管理；
 - 活动发布和下线；
 - 单次抽奖；
 - 十连抽；
 - 用户每日次数限制；
 - 抽奖请求幂等；
 - Redis 原子库存扣减；
 - 抽奖订单和中奖记录；
 - 用户权益查询；
 - 活动配置缓存；
 - 缓存穿透处理；
 - 全局异常处理；
 - 参数校验；
 - Swagger/OpenAPI 接口文档；
 - Docker Compose 一键启动；
 - Linux 部署；
 - 基础压力测试。
-

3.2 第一阶段暂不包含内容

以下功能不进入第一阶段：

- 真实支付；
- 用户充值；
- 现金奖品；
- 提现功能；
- 实物物流配送；
- 多级分销；
- 微服务拆分；
- Elasticsearch；
- Kafka、RabbitMQ、RocketMQ；
- Kubernetes；
- 分库分表；
- 推荐系统；
- 复杂数据仓库；

- 多租户系统；
- 完整风控系统；
- 前端复杂抽奖动画。

Redis Stream、排行榜和保底机制可以作为加分功能，不影响第一阶段验收。

4. 用户角色分析

系统包含三类用户。

4.1 活动用户

角色编码：

PLAYER

活动用户可以：

- 登录和退出；
- 查看正在进行的活动；
- 查看活动详情；
- 查看剩余参与次数；
- 参与单次抽奖；
- 参与十连抽；
- 查询个人抽奖记录；
- 查询个人已获得权益；
- 查询活动排行榜。

活动用户不能：

- 创建活动；
 - 修改奖品；
 - 查看其他用户记录；
 - 修改中奖概率；
 - 修改活动库存。
-

4.2 运营人员

角色编码：

OPERATOR

运营人员可以：

- 创建和修改奖品；

- 创建和修改营销活动；
- 为活动配置奖品；
- 配置奖品库存；
- 配置奖品中奖权重；
- 设置活动时间；
- 设置每日参与次数；
- 发布活动；
- 关闭活动；
- 查询活动参与记录；
- 查询库存消耗情况；
- 查询中奖记录。

运营人员不能：

- 管理系统管理员；
- 修改系统核心权限；
- 删除系统内置角色。

4.3 系统管理员

角色编码：

ADMIN

系统管理员拥有：

- 用户管理权限；
- 角色管理权限；
- 权限管理权限；
- 用户角色分配权限；
- 角色权限分配权限；
- 活动管理权限；
- 奖品管理权限；
- 系统日志查看权限。

5. 业务术语

5.1 营销活动

营销活动是用户参与抽奖的业务载体。

一个活动包含：

- 活动名称；
- 活动说明；
- 开始时间；

- 结束时间；
 - 活动状态；
 - 每日参与次数；
 - 奖品列表；
 - 奖品库存；
 - 奖品中奖权重。
-

5.2 奖品

奖品是活动中可以被用户抽中的权益。

第一阶段支持以下奖品类型：

- 优惠券；
- 积分；
- 会员权益；
- 虚拟兑换码；
- 虚拟礼品；
- 模拟实物奖品。

第一阶段只记录权益发放结果，不处理真实物流和支付。

5.3 抽奖订单

每一次用户抽奖请求对应一个抽奖订单。

抽奖订单用于记录：

- 请求唯一标识；
 - 用户；
 - 活动；
 - 抽奖次数；
 - 处理状态；
 - 创建时间；
 - 完成时间。
-

5.4 中奖记录

中奖记录记录一次具体的抽奖结果。

单抽产生一条中奖记录。

十连抽产生十条中奖记录，并归属于同一个抽奖订单。

5.5 用户权益

用户权益表示已经发放给用户的奖品。

例如：

- 用户拥有一张优惠券；
- 用户获得 100 积分；
- 用户获得 7 天会员；
- 用户获得一个虚拟兑换码。

6. 核心业务规则

6.1 活动状态

活动状态定义如下：

状态	说明
DRAFT	草稿，尚未发布
PUBLISHED	已发布但尚未开始
RUNNING	活动进行中
ENDED	活动已经结束
DISABLED	活动被人工关闭

状态变化建议如下：

DRAFT → PUBLISHED → RUNNING → ENDED
↘ DISABLED

业务限制：

1. 草稿状态可以修改活动信息。
2. 活动发布前必须配置至少一个奖品。
3. 活动发布前必须完成权重和库存配置。
4. 活动开始后不能直接修改中奖概率。
5. 活动结束后不能继续抽奖。
6. 被关闭的活动不能继续参与。
7. 活动开始时间必须早于结束时间。

6.2 奖品权重

中奖概率采用整数权重实现。

示例：

奖品	权重
特等奖	10
一等奖	90
二等奖	900
普通奖励	9000

总权重：

10000

对应概率：

- 特等奖：0.1%
- 一等奖：0.9%
- 二等奖：9%
- 普通奖励：90%

系统随机生成一个位于 1 到总权重之间的整数，根据权重区间选择奖品。

6.3 库存规则

1. 有限奖品必须配置库存。
2. 库存不能小于 0。
3. 活动发布时将奖品库存加载到 Redis。
4. 抽奖时优先在 Redis 中进行库存校验和扣减。
5. Redis 库存扣减必须使用 Lua 脚本保证原子性。
6. 数据库中中奖记录数量不能超过奖品总库存。
7. 奖品库存不足时不能继续发放该奖品。
8. 抽中奖品但库存不足时，系统最多重新抽取三次。
9. 多次重试仍无法获得可用奖品时，返回默认保底奖品或抽奖失败。

6.4 用户次数限制

每个活动可以设置用户每日最大参与次数。

示例：

每日最多参与 5 次

Redis Key：


```
lucky:daily:{activityId}:{userId}:{yyyyMMdd}
```

规则：

1. 每次成功受理抽奖请求时，参与次数加一。
2. 超过每日限制时拒绝请求。
3. 当天次数 Key 在当天结束后自动过期。
4. 十连抽按照十次参与次数计算。
5. 次数校验和次数增加应尽量在 Lua 脚本中原子执行。

6.5 幂等规则

每次抽奖请求必须携带唯一的 `requestId`。

示例：

```
{
  "requestId": "e21a3109-c262-4551-a647-41aee5206f9f",
  "drawCount": 1
}
```

系统通过以下方式防止重复处理：

1. Redis 使用 `SET NX` 判断请求是否首次出现。
2. 抽奖订单表对 `request_id` 建立唯一索引。
3. 已完成请求再次提交时，可以返回原有处理结果。
4. 正在处理中的请求再次提交时，返回“请求处理中”。
5. 同一个 `requestId` 不能创建多个抽奖订单。

6.6 并发规则

系统使用不同机制解决不同并发问题。

用户级分布式锁

Redis Key：

```
lock:draw:user:{userId}
```

作用：

- 防止同一用户同时提交多个抽奖请求；
- 防止重复扣减用户参与次数；
- 避免用户连续点击造成并行业务处理。

Lua 库存扣减

作用：

- 保证库存校验和扣减原子执行；
- 防止多个用户同时抽中有限奖品时发生超卖。

请求幂等

作用：

- 防止网络重试；
- 防止浏览器重复提交；
- 防止客户端重复发送相同业务请求。

不能使用活动级全局锁，因为全局锁会使所有用户串行执行，严重降低系统吞吐量。

7. 功能需求

7.1 认证模块

7.1.1 用户登录

用户通过用户名和密码登录。

请求示例：

```
POST /api/auth/login
```

```
{
  "username": "player01",
  "password": "123456"
}
```

登录成功后：

1. 校验用户名和密码；
2. 校验用户状态；
3. 生成 Session ID；
4. 将用户信息、角色和权限保存到 Redis；
5. 生成 JWT；
6. 返回 JWT 和用户基础信息。

返回示例：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "accessToken": "jwt-token",
    "expiresIn": 7200,
    "user": {
      "id": 1001,
      "username": "player01",
      "roles": ["PLAYER"]
    }
  }
}
```

7.1.2 用户退出

`POST /api/auth/logout`

退出时删除 Redis 中的登录状态。

JWT 尚未过期但 Redis Session 已删除时，也必须视为未登录。

7.1.3 身份认证

除登录接口和公开活动查询接口外，其他接口必须登录。

认证流程：

```
读取 Authorization
    ↓
解析 JWT
    ↓
校验 JWT 签名和过期时间
    ↓
读取 Session ID
    ↓
查询 Redis 登录状态
    ↓
认证通过
```

7.2 RBAC 权限模块

系统采用基于角色的权限控制模型。

关系如下：

用户 $\longleftrightarrow$ 用户角色 $\longleftrightarrow$ 角色 $\longleftrightarrow$ 角色权限 $\longleftrightarrow$ 权限

主要功能：

- 用户分页查询；
- 创建用户；
- 禁用用户；
- 查询角色；
- 创建角色；
- 给用户分配角色；
- 查询权限；
- 给角色分配权限。

权限编码示例：

```
activity:view  
activity:create  
activity:update  
activity:publish  
activity:disable  
prize:view  
prize:create  
prize:update  
draw:query  
user:manage  
role:manage  
permission:manage
```

7.3 奖品管理模块

运营人员可以：

- 创建奖品；
- 修改奖品；
- 禁用奖品；
- 分页查询奖品；
- 根据名称、类型和状态筛选奖品。

奖品字段包括：

- 奖品名称；
- 奖品类型；
- 奖品等级；
- 图片地址；
- 奖品说明；
- 是否可叠加；
- 奖品状态。

奖品已经被活动使用时，不建议物理删除，应采用禁用或逻辑删除。

7.4 活动管理模块

运营人员可以：

- 创建活动；
- 修改活动；
- 查询活动详情；
- 分页查询活动；
- 配置活动奖品；
- 配置奖品权重；
- 配置奖品库存；
- 发布活动；
- 关闭活动。

活动分页查询支持：

- 活动名称；
- 活动状态；
- 开始时间；
- 结束时间；
- 创建人。

发布活动前必须校验：

1. 活动名称不为空；
 2. 开始时间早于结束时间；
 3. 至少配置一个奖品；
 4. 所有奖品权重大于 0；
 5. 权重总和大于 0；
 6. 有限奖品库存大于或等于 0；
 7. 活动不处于运行或结束状态。
-

7.5 抽奖模块

7.5.1 查询活动列表

```
GET /api/activities
```

只返回：

- 已发布；
- 正在进行；
- 未被禁用的活动。

支持分页查询。

7.5.2 查询活动详情

```
GET /api/activities/{activityId}
```

返回：

- 活动名称；
- 活动说明；
- 活动时间；
- 剩余时间；
- 每日次数；
- 用户剩余次数；
- 奖品展示列表。

不向用户返回精确中奖权重，可以只显示奖品等级和说明。

7.5.3 单次抽奖

```
POST /api/activities/{activityId}/draw
```

请求：

```
{
  "requestId": "e21a3109-c262-4551-a647-41aee5206f9f",
  "drawCount": 1
}
```

处理流程：

1. 校验登录状态；
2. 校验参数；
3. 校验活动状态和时间；
4. 校验请求幂等；
5. 获取用户级分布式锁；
6. 查询用户剩余次数；
7. 查询活动配置缓存；
8. 根据权重选择奖品；
9. 使用 Lua 扣减奖品库存；
10. 使用事务创建抽奖订单；
11. 保存中奖记录；
12. 发放用户权益；
13. 更新抽奖订单状态；
14. 返回抽奖结果；
15. 释放分布式锁。

7.5.4 十连抽

十连抽请求：

```
{
  "requestId": "36ed5574-364c-4519-b350-9514bbdeef63",
  "drawCount": 10
}
```

规则：

1. `drawCount` 只能为 1 或 10。
2. 用户剩余次数必须不少于 10。
3. 十连抽对应一个抽奖订单。
4. 十连抽产生十条中奖记录。
5. 十条记录必须使用同一个数据库事务保存。
6. 中途发生异常时，整个数据库事务回滚。
7. Redis 已经扣减的库存需要进行补偿或记录待补偿任务。

7.6 抽奖记录模块

活动用户可以查询自己的抽奖记录。

```
GET /api/me/draw-records
```

支持条件：

- 活动名称；
- 奖品名称；
- 奖品等级；
- 开始时间；
- 结束时间；
- 页码；
- 每页数量。

运营人员可以查询全部用户记录。

```
GET /api/admin/draw-records
```

运营查询支持：

- 用户名；
- 活动名称；
- 奖品名称；
- 请求 ID；
- 抽奖状态；
- 时间范围。

7.7 用户权益模块

活动用户可以查询自己已经获得的权益。

```
GET /api/me/benefits
```

返回：

- 奖品名称；
- 奖品类型；
- 数量；
- 获得时间；
- 使用状态；
- 有效期。

对于可叠加奖品，可以将相同用户和相同奖品的数量累加。

8. 非功能需求

8.1 性能需求

第一阶段不规定固定的生产级 QPS，但必须完成本地压力测试。

压测至少覆盖：

- 100 个并发用户；
- 500 个并发用户；
- 同一用户并发请求；
- 多用户同时抽奖；
- 相同 `requestId` 重复提交；
- 有限奖品库存并发扣减。

压测后必须验证：

- Redis 库存不小于 0；
 - 数据库中奖记录不超过总库存；
 - 同一请求不会产生多个订单；
 - 同一用户不会超过每日参与次数；
 - 失败请求不会产生完整权益记录。
-

8.2 可用性需求

1. 接口返回统一格式。
 2. 参数错误需要返回明确提示。
 3. 未知异常不能将系统堆栈返回给前端。
 4. Redis 短暂异常时必须记录错误日志。
 5. 数据库异常时事务必须回滚。
 6. 应用启动失败时日志中必须包含明确原因。
-

8.3 安全需求

1. 密码必须加密保存。
 2. JWT 密钥不能写死在公开代码中。
 3. 数据库密码和 Redis 密码使用环境变量配置。
 4. 普通用户不能访问后台管理接口。
 5. 运营人员不能修改管理员权限。
 6. 日志中不能打印用户密码和完整 Token。
 7. 所有后台修改操作必须进行权限校验。
 8. 分页参数必须设置最大值，避免一次查询过多数据。
-

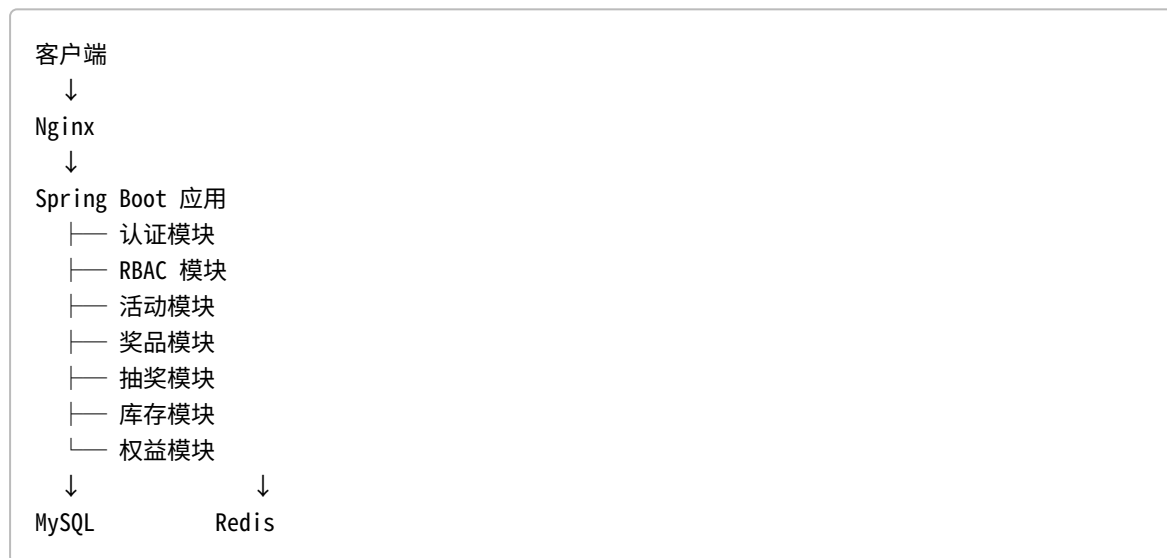
8.4 可维护性需求

1. Controller 不直接操作 Mapper。
 2. 业务逻辑放在 Service。
 3. DTO、VO 和数据库实体分离。
 4. 使用统一异常类型。
 5. 使用统一返回对象。
 6. 关键业务方法必须包含日志。
 7. Redis Key 统一管理。
 8. 业务状态使用枚举。
 9. 数据库字段禁止使用无意义缩写。
 10. 核心接口必须编写注释和 OpenAPI 文档。
-

9. 总体架构设计

9.1 架构模式

第一阶段采用单体模块化架构。



不拆微服务的原因：

- 第一阶段开发周期只有 1~2 周；
 - 当前目标是验证核心业务能力；
 - 微服务会引入注册中心、网关、配置中心和分布式事务；
 - 过早拆分会增加开发和排障成本；
 - 单体模块化已经可以验证大部分后端能力。
-

9.2 技术栈

建议技术栈：

类型	技术
编程语言	Java 21
开发框架	Spring Boot
Web	Spring MVC
ORM	MyBatis 或 MyBatis-Plus
数据库	MySQL 8
缓存	Redis
分布式锁	Redisson
认证	JWT + Redis Session
参数校验	Jakarta Bean Validation
接口文档	SpringDoc OpenAPI
容器化	Docker、Docker Compose
构建工具	Maven
测试	JUnit 5、Mockito
压力测试	JMeter
部署环境	Linux

MyBatis 和 MyBatis-Plus 选择一个即可，不建议同时混用两套风格。

10. 模块设计

建议模块结构：

```
com.luckyhub
├── common
│   ├── result
│   ├── exception
│   ├── enums
│   ├── validation
│   └── util
├── auth
│   ├── controller
│   ├── service
│   ├── dto
│   └── security
├── rbac
│   ├── controller
│   ├── service
│   └── mapper
```

```

|   ├── entity
|   ├── dto
|   └── vo
├── prize
├── activity
├── lottery
├── inventory
├── benefit
├── config
└── LuckyHubApplication

```

模块职责：

模块	职责
common	统一返回、异常、工具类、枚举
auth	登录、退出、JWT、Redis Session
rbac	用户、角色、权限
prize	奖品基础信息管理
activity	活动及活动奖品配置
lottery	抽奖订单、权重随机、中奖记录
inventory	Redis 库存、Lua 脚本、库存补偿
benefit	用户权益发放和查询
config	Redis、Redisson、OpenAPI、MVC 配置

11. 数据库设计

11.1 系统用户表

sys_user

字段	类型	说明
id	BIGINT	主键
username	VARCHAR(50)	用户名
password	VARCHAR(255)	加密密码
nickname	VARCHAR(50)	昵称
status	TINYINT	用户状态
created_at	DATETIME	创建时间

字段	类型	说明
updated_at	DATETIME	修改时间

约束：

- `username` 唯一；
- 密码不能明文保存。

11.2 角色表

sys_role

字段	类型	说明
id	BIGINT	主键
role_code	VARCHAR(50)	角色编码
role_name	VARCHAR(50)	角色名称
status	TINYINT	状态
created_at	DATETIME	创建时间

`role_code` 必须唯一。

11.3 权限表

sys_permission

字段	类型	说明
id	BIGINT	主键
permission_code	VARCHAR(100)	权限编码
permission_name	VARCHAR(100)	权限名称
created_at	DATETIME	创建时间

11.4 用户角色表

sys_user_role

字段	类型	说明
user_id	BIGINT	用户 ID

字段	类型	说明
role_id	BIGINT	角色 ID

联合唯一索引：

user_id + role_id

11.5 角色权限表

sys_role_permission

字段	类型	说明
role_id	BIGINT	角色 ID
permission_id	BIGINT	权限 ID

联合唯一索引：

role_id + permission_id

11.6 奖品表

marketing_prize

字段	类型	说明
id	BIGINT	主键
prize_name	VARCHAR(100)	奖品名称
prize_type	VARCHAR(30)	奖品类型
prize_level	VARCHAR(30)	奖品等级
image_url	VARCHAR(500)	图片地址
description	VARCHAR(500)	奖品说明
stackable	TINYINT	是否可叠加
status	TINYINT	状态
created_at	DATETIME	创建时间
updated_at	DATETIME	修改时间

11.7 活动表

marketing_activity

字段	类型	说明
id	BIGINT	主键
activity_name	VARCHAR(100)	活动名称
description	VARCHAR(1000)	活动说明
status	VARCHAR(30)	活动状态
start_time	DATETIME	开始时间
end_time	DATETIME	结束时间
daily_limit	INT	每日次数
created_by	BIGINT	创建人
created_at	DATETIME	创建时间
updated_at	DATETIME	修改时间

索引建议：

- status；
- start_time；
- end_time；
- activity_name。

11.8 活动奖品表

marketing_activity_prize

字段	类型	说明
id	BIGINT	主键
activity_id	BIGINT	活动 ID
prize_id	BIGINT	奖品 ID
weight	INT	中奖权重
total_stock	INT	总库存
remaining_stock	INT	数据库剩余库存
sort_order	INT	展示顺序
created_at	DATETIME	创建时间
updated_at	DATETIME	修改时间

联合唯一索引：

activity_id + prize_id

11.9 抽奖订单表

lottery_draw_order

字段	类型	说明
id	BIGINT	主键
request_id	VARCHAR(64)	请求唯一标识
user_id	BIGINT	用户 ID
activity_id	BIGINT	活动 ID
draw_count	INT	抽奖次数
status	VARCHAR(30)	订单状态
fail_reason	VARCHAR(500)	失败原因
created_at	DATETIME	创建时间
completed_at	DATETIME	完成时间

request_id 必须建立唯一索引。

订单状态：

PROCESSING
SUCCESS
FAILED

11.10 中奖记录表

lottery_draw_record

字段	类型	说明
id	BIGINT	主键
order_id	BIGINT	抽奖订单 ID
request_id	VARCHAR(64)	请求 ID
sequence_no	INT	结果序号

字段	类型	说明
user_id	BIGINT	用户 ID
activity_id	BIGINT	活动 ID
prize_id	BIGINT	奖品 ID
draw_time	DATETIME	抽奖时间
created_at	DATETIME	创建时间

联合唯一索引：

```
request_id + sequence_no
```

11.11 用户权益表

user_benefit

字段	类型	说明
id	BIGINT	主键
user_id	BIGINT	用户 ID
prize_id	BIGINT	奖品 ID
quantity	INT	数量
status	VARCHAR(30)	权益状态
obtained_at	DATETIME	获得时间
expire_at	DATETIME	过期时间
created_at	DATETIME	创建时间
updated_at	DATETIME	修改时间

如果奖品可叠加，可以使用：

```
user_id + prize_id
```

作为联合唯一索引。

12. Redis 设计

12.1 Redis Key 规范

统一前缀：

lucky:

Key 设计：

Key	数据结构	说明
lucky:session:{sessionId}	String/Hash	登录状态
lucky:activity:{activityId}	String	活动详情缓存
lucky:activity:null:{activityId}	String	空值缓存
lucky:stock:{activityId}	Hash	活动奖品库存
lucky:daily:{activityId}:{userId}:{date}	String	每日次数
lucky:idempotent:{requestId}	String	幂等标识
lucky:lock:draw:user:{userId}	Lock	用户分布式锁
lucky:rank:{activityId}	ZSet	活动排行榜
lucky:pity:{activityId}:{userId}	String	保底计数

12.2 活动缓存

查询活动详情：

查询 Redis
↓ 未命中
查询 MySQL
↓
写入 Redis

数据库不存在时缓存空值。

空值缓存过期时间：

2~5 分钟

正常活动缓存过期时间建议增加随机值，避免大量缓存同时过期。

更新活动时：

```
更新数据库
↓
删除活动缓存
```

12.3 库存缓存

活动发布时，将库存加载到 Redis Hash：

```
lucky:stock:{activityId}
```

示例：

```
prize:101 → 10
prize:102 → 100
prize:103 → 1000
```

扣减库存必须使用 Lua 脚本。

Lua 脚本需要完成：

1. 读取奖品库存；
2. 判断库存是否存在；
3. 判断库存是否大于 0；
4. 扣减库存；
5. 返回剩余库存。

12.4 幂等缓存

处理前执行：

```
SET lucky:idempotent:{requestId} PROCESSING NX EX 60
```

处理成功后更新为：

```
SUCCESS
```

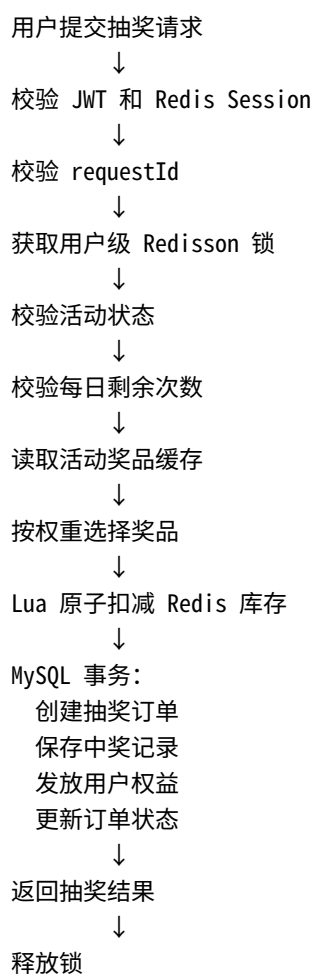
并延长过期时间。

处理失败时根据失败类型决定：

- 删除幂等 Key，允许重试；
- 保留失败状态，防止重复处理；
- 返回原订单状态。

13. 抽奖核心流程设计

同步版本流程：



异常处理：

- Redis 扣减失败：重新抽奖或返回库存不足；
- 数据库事务失败：回滚数据库并尝试补偿 Redis 库存；
- 重复请求：返回原有订单结果；
- 获取锁失败：返回操作频繁；
- 活动结束：拒绝抽奖；
- 用户次数不足：拒绝抽奖。

14. 事务设计

以下操作必须放在同一个 MySQL 事务中：

创建抽奖订单
保存中奖记录
更新或插入用户权益
更新抽奖订单状态

事务失败时：

- 抽奖订单不应处于成功状态；
- 中奖记录不能单独存在；
- 用户权益不能部分发放；
- 十连抽不能只保存部分记录。

Redis 操作不属于 MySQL 本地事务。

第一阶段采用：

Redis 预扣库存 + MySQL 本地事务 + 失败补偿

并在项目 README 中说明该方案仍存在极端情况下的数据一致性风险。

后续可以通过以下方式优化：

- Redis Stream；
- 消息队列；
- 本地消息表；
- 定时对账；
- 补偿任务。

15. 接口设计概览

15.1 认证接口

POST /api/auth/login
POST /api/auth/logout
GET /api/users/me

15.2 用户端接口

```
GET /api/activities
GET /api/activities/{id}
POST /api/activities/{id}/draw
GET /api/me/draw-records
GET /api/me/benefits
GET /api/activities/{id}/ranking
```

15.3 奖品管理接口

```
POST /api/admin/prizes
PUT /api/admin/prizes/{id}
GET /api/admin/prizes/{id}
GET /api/admin/prizes
DELETE /api/admin/prizes/{id}
```

删除建议实际实现为逻辑删除或禁用。

15.4 活动管理接口

```
POST /api/admin/activities
PUT /api/admin/activities/{id}
GET /api/admin/activities/{id}
GET /api/admin/activities

POST /api/admin/activities/{id}/prizes
PUT /api/admin/activities/{id}/prizes/{prizeId}

POST /api/admin/activities/{id}/publish
POST /api/admin/activities/{id}/disable
```

15.5 权限管理接口

```
GET /api/admin/users
POST /api/admin/users/{id}/roles

GET /api/admin/roles
POST /api/admin/roles
POST /api/admin/roles/{id}/permissions

GET /api/admin/permissions
```

16. 统一返回格式

成功返回：

```
{
  "code": 0,
  "message": "success",
  "data": {}
}
```

失败返回：

```
{
  "code": 40001,
  "message": "今日参与次数已用完",
  "data": null,
  "timestamp": 1784000000000,
  "requestId": "e21a3109-c262-4551-a647-41aee5206f9f"
}
```

建议错误码分类：

范围	类型
10000~19999	系统错误
20000~29999	认证和权限错误
30000~39999	参数错误
40000~49999	活动和抽奖错误
50000~59999	库存和权益错误

17. 异常设计

建议异常类型：

```
BusinessException
NotFoundException
UnauthorizedException
ForbiddenException
DuplicateRequestException
ActivityNotAvailableException
DailyLimitExceededException
```

```
StockNotEnoughException  
LockAcquireFailedException
```

全局异常处理器：

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
}
```

需要处理：

- 参数校验异常；
- JSON 格式异常；
- 业务异常；
- 唯一索引冲突；
- 数据库异常；
- Redis 异常；
- 未知异常。

未知异常只返回通用错误信息，完整堆栈写入日志。

18. 参数校验要求

创建活动校验：

- 活动名称不能为空；
- 开始时间不能为空；
- 结束时间不能为空；
- 开始时间必须早于结束时间；
- 每日次数必须大于 0；
- 活动说明长度不能超限。

配置奖品时校验：

- 奖品 ID 必须存在；
- 权重必须大于 0；
- 库存不能小于 0；
- 同一活动不能重复添加同一奖品。

抽奖时校验：

- `requestId` 不能为空；
 - `requestId` 长度不能超限；
 - `drawCount` 只能为 1 或 10；
 - 活动 ID 必须存在；
 - 用户必须登录。
-

19. 日志设计

建议使用结构化日志。

核心日志包括：

- 用户登录成功和失败；
- 活动创建、修改、发布和关闭；
- 奖品库存初始化；
- 抽奖请求开始；
- 幂等校验失败；
- 分布式锁获取失败；
- Redis 库存扣减成功和失败；
- MySQL 事务失败；
- 库存补偿结果；
- 抽奖订单完成；
- 未知系统异常。

日志中建议包含：

```
requestId  
userId  
activityId  
orderId  
prizeId
```

禁止记录：

- 用户密码；
- 完整 JWT；
- 数据库密码；
- Redis 密码。

20. Docker 部署设计

Docker Compose 至少包含：

```
luckyhub-app  
mysql  
redis
```

目录建议：

```
luckyhub  
├── Dockerfile
```

```
├─ docker-compose.yml
├─ sql
│   └─ init.sql
├─ src
├─ pom.xml
└─ README.md
```

启动命令：

```
docker compose up -d
```

关闭命令：

```
docker compose down
```

环境变量包括：

```
MYSQL_HOST
MYSQL_PORT
MYSQL_DATABASE
MYSQL_USERNAME
MYSQL_PASSWORD
REDIS_HOST
REDIS_PORT
REDIS_PASSWORD
JWT_SECRET
```

21. 测试需求

21.1 功能测试

至少测试：

- 正确账号登录；
- 错误密码登录；
- 退出后 Token 失效；
- 普通用户访问后台接口；
- 创建奖品；
- 创建活动；
- 活动未配置奖品时发布；
- 活动开始后抽奖；
- 活动结束后抽奖；
- 每日次数超限；
- 奖品库存为零；

- 单抽；
- 十连抽；
- 查询中奖记录；
- 查询用户权益。

21.2 并发测试

至少测试：

1. 1000 个请求争抢库存为 10 的奖品。
2. 相同 `requestId` 重复提交 100 次。
3. 同一个用户并发发送 20 个请求。
4. 多个用户并发抽奖。
5. Redis 中删除活动缓存后并发查询。
6. 查询不存在的活动 1000 次。
7. 数据库事务中主动抛出异常。
8. Redis 扣减后数据库保存失败。

21.3 数据正确性验收

必须满足：

Redis 库存 ≥ 0
数据库中中奖数量 $\leq$ 初始库存
同一 `requestId` 只有一个抽奖订单
失败订单不产生完整用户权益
用户每日参与次数不超过限制
十连抽不会只保存部分结果

22. 项目验收标准

总分 100 分。

模块	分数
用户登录、JWT、Redis Session	10
RBAC 权限	10
奖品管理	5
活动管理	10
分页、查询、参数校验	5
全局异常处理	5

模块	分数
活动缓存	5
缓存穿透处理	5
权重抽奖	10
Redis Lua 库存扣减	10
请求幂等	5
Redisson 用户锁	5
MySQL 事务	5
Docker Compose	5
Linux 部署	5

达到 70 分视为第一阶段完成。

加分项：

功能	加分
Redis Stream 异步落库	10
排行榜	5
保底机制	5
单元测试	5
压测报告	5
库存自动对账	5

23. 开发计划

第 1 天

- 初始化 Spring Boot 项目；
- 创建数据库；
- 配置 MySQL 和 Redis；
- 统一返回格式；
- 全局异常处理；
- OpenAPI 文档。

第 2 天

- 用户登录；
- JWT；

- Redis Session;
- 登录拦截器;
- RBAC 权限。

第 3 天

- 奖品 CRUD;
- 分页查询;
- 条件查询;
- 参数校验。

第 4 天

- 活动 CRUD;
- 活动奖品配置;
- 权重配置;
- 库存配置;
- 活动发布。

第 5 天

- 活动缓存;
- 空值缓存;
- 缓存更新;
- Redis 库存初始化。

第 6 天

- 权重抽奖;
- Lua 扣减库存;
- 单抽;
- 抽奖订单;
- 中奖记录;
- 用户权益。

第 7 天

- requestId 幂等;
- Redisson 用户锁;
- 每日次数限制;
- 重复请求测试。

第 8 天

- 十连抽;
- 抽奖记录查询;
- 用户权益查询;
- 后台抽奖记录查询。

第 9 天

- Dockerfile;
- Docker Compose;
- 初始化 SQL;
- 环境变量配置;
- 一键启动。

第 10 天

- Linux 部署;
- JMeter 压测;
- 并发问题修复;
- README;
- 项目总结。

24. 后续迭代方向

V1.1

- Redis Stream 异步落库;
- 消费者组;
- Pending List;
- 消息重试;
- 消费幂等。

V1.2

- ZSet 活动排行榜;
- 用户幸运值;
- 抽奖保底机制;
- 活动实时统计。

V2.0

- Spring Cloud Gateway;
- Nacos;
- OpenFeign;
- Sentinel;
- RocketMQ;
- 服务拆分。

服务可以拆分为：

```
auth-service
activity-service
lottery-service
```

benefit-service
gateway-service

V3.0

- 风控和黑名单;
- IP 限流;
- 设备限制;
- 库存对账;
- Prometheus;
- Grafana;
- 链路追踪;
- Elasticsearch 查询。

25. 简历项目描述参考

项目名称:

LuckyHub 高并发营销抽奖与权益发放平台

项目简介:

基于 Spring Boot、MySQL、Redis 和 Redisson 开发的营销抽奖平台，支持营销活动配置、奖品权重与库存管理、用户参与限制、抽奖记录、权益发放和 RBAC 权限管理。针对营销活动高峰期的并发请求，实现缓存、库存防超卖、接口幂等、用户级分布式锁和数据库事务处理。

技术亮点:

1. 使用 Redis 缓存活动和奖品配置，通过缓存空值降低恶意请求造成的缓存穿透。
2. 使用 Lua 脚本原子完成库存校验和扣减，防止高并发场景下奖品库存超卖。
3. 使用客户端 `requestId`、Redis `SET NX` 和 MySQL 唯一索引构建多层幂等机制。
4. 使用 Redisson 用户级分布式锁限制同一用户并发操作，避免使用全局锁降低系统吞吐量。
5. 使用 MySQL 事务统一保存抽奖订单、中奖记录和用户权益，保证核心业务数据完整性。
6. 使用 JWT 和 Redis Session 实现登录认证，并基于 RBAC 模型实现后台接口权限控制。
7. 使用 Docker Compose 编排应用、MySQL 和 Redis，实现开发和部署环境一键启动。
8. 使用 JMeter 验证高并发环境下库存不为负、中奖数量不超过库存和重复请求不重复处理。

压测数据必须在完成实际测试后再补充，不能编造。